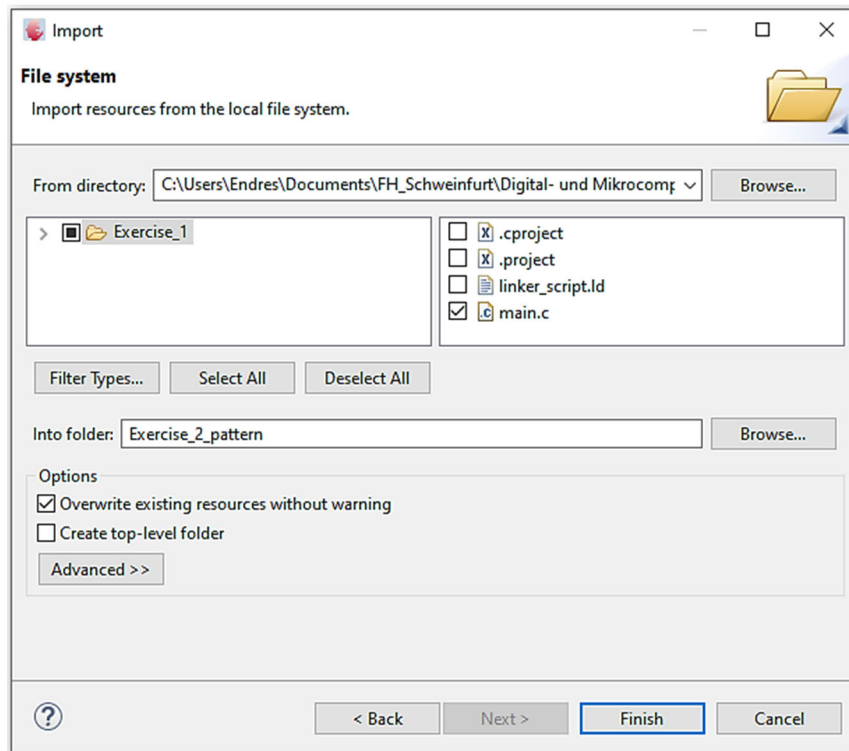


XMC1200 Boot Kit – Debugging

1 Anzeige verschiedener Muster

Erzeugen Sie ein neues Projekt „Exercise_2_pattern“ analog zum Übungsblatt 1. Mit der rechten Maustaste gehen Sie auf den Projektnamen im Projects-Browser, und wählen  Import... . Mit der Option „General → File System“ wählen Sie das File „Main.c“ aus der Übung 1 (Exercise_1) aus:



Ändern Sie den Code in Main.c ab, so dass abwechselnd die Muster „10101“ und „01010“ an den LEDs angezeigt werden.

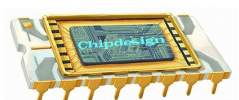
Starten Sie den Ihr Programm mit dem Debugger, wie in Übung 1 beschrieben. Falls alles richtig ist, sollten alle fünf LEDs leuchten. Weshalb?

2 Verwendung des Debuggers

Halten Sie jetzt den Prozessor mit „Debug → Suspend “ an. Mit Hilfe der Taste Debug →  Step Into, bzw. F5 können Sie das Programm zeilenweise laufen lassen. Versuchen Sie so zu erkennen, ob Ihr Code richtig arbeitet, und machen Sie sich mit der Funktion des Debuggers vertraut.

3 Einfügen einer Warteschleife

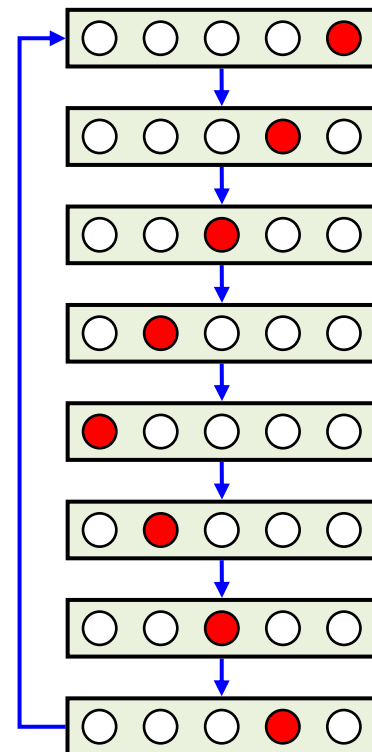
Erzeugen Sie ein Unterprogramm mit dem Namen „wait“, in dem der Prozessor bis 500.000 zählt. Rufen Sie dieses Unterprogramm nach jedem Muster auf, so dass ein entsprechendes Blinklicht sichtbar wird.



4 Erstellen eines Lauflichtes

Erzeugen Sie analog ein neues Projekt mit dem Namen „Exercise_2_runningLight1“. In diesem Projekt soll ähnlich dem Beispiel im Skript ein 5-Kanal Lauflicht programmiert werden. Um den eigentlichen Algorithmus einfach zu halten, schreiben Sie zuerst ein Unterprogram namens `writeLED(int value)`, welches ein 5-Bit Muster direkt auf die Anzeige überträgt. Der Aufruf `writeLED(0x15)` soll also z.B. das Muster „10101“ anzeigen.

Anschließend programmieren Sie das Lauflicht entsprechend der Abbildung rechts:



5 Ein weiteres Lauflichtes

Erzeugen Sie analog ein neues Projekt mit dem Namen „Exercise_2_runningLight2“ und folgendem Muster:

6 Assembler-Code des Lauflichtes (Tricky)

Betrachten Sie den Assembler-Code des Lauflichtes, insbesondere die Realisierung der Warteschleife `wait()`. Der Code scheint umständlicher realisiert zu sein als im Skript vorgestellt. Weshalb? Wie lässt sich das C-Programm abändern, damit ein Maschinencode entsteht, der ähnlich zur Vorlesung ist?

